ENTERPRISE SYSTEM ARCHITECTURE & ENGINEERING SPECIFICATIONS

CropDeal Microservices Architecture & Design

Comprehensive Technical Reference Manual with Architectural Topology, UML Domain Class Specifications, and Entity-Relationship (ER) Data Models.

System Type:	B2B Agricultural Marketplace Platform	**Build Tool:**	Apache Maven & Docker Compose
Framework:	Spring Boot 3.3.13 | Java 21 LTS	**Service Discovery:**	Spring Cloud Netflix Eureka (:8761)
API Gateway:	Spring Cloud Gateway (:8080)	**Configuration:**	100% Centralized Config Server (:8888)
Message Broker:	RabbitMQ 3.x (Direct/Topic AMQP)	**Relational DB:**	Distributed MySQL Schemas
Frontend:	Angular 17 Responsive Portals	**Documentation:**	SpringDoc OpenAPI v3 / Swagger UI

Direct Trade Engine

Eliminates middleman exploitation by connecting verified farmers directly to bulk commercial dealers.

Multi-Model Commerce

Supports Direct Buy, Real-time Competitive Auctions, and Bilateral Counter-Offer Negotiations.

Escrow Digital Wallet

Double-spend protected escrow funds holding until verified OTP delivery confirmation.

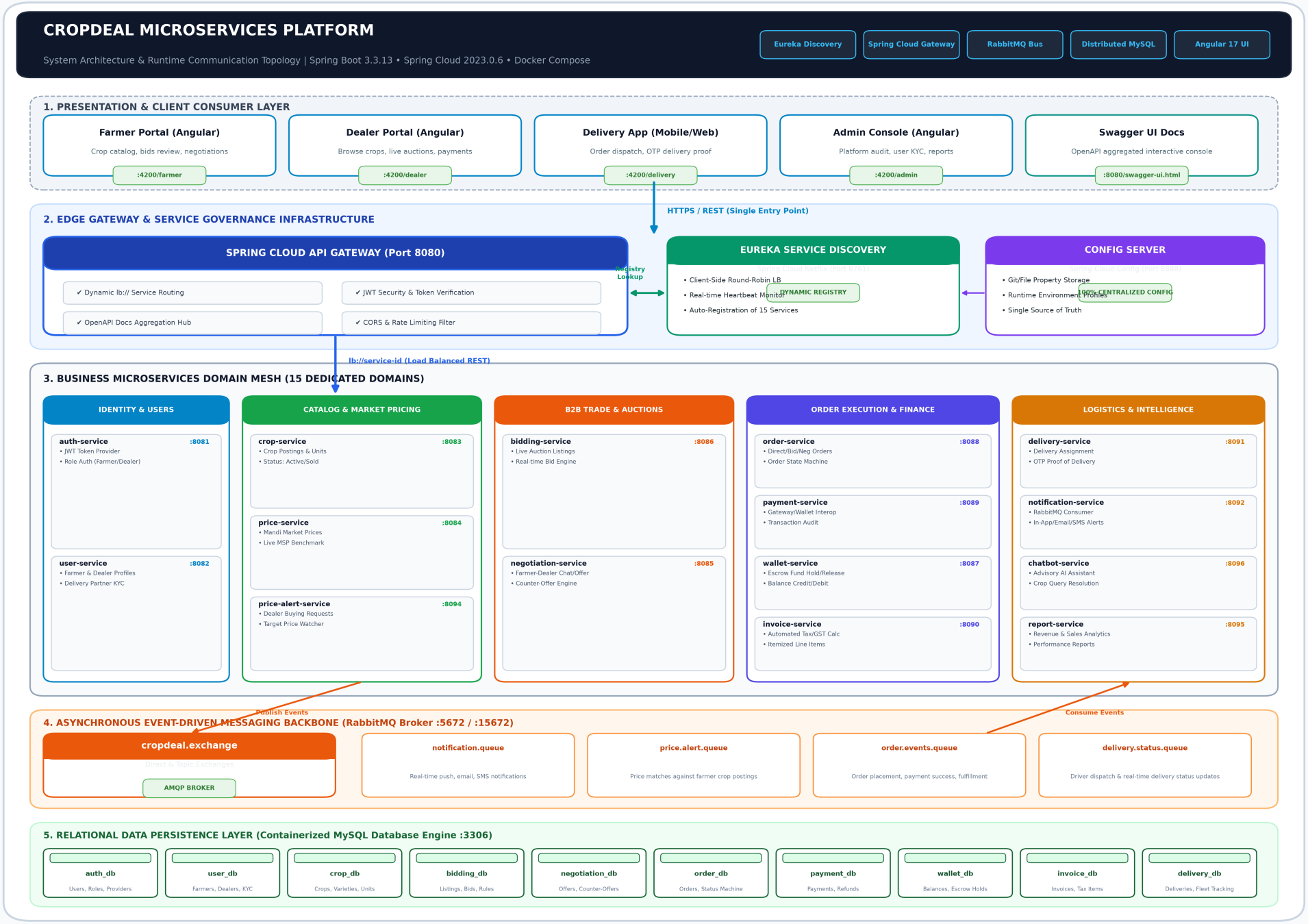
1. Executive Summary & Platform Overview

CropDeal is an enterprise-grade distributed microservices platform engineered to eliminate exploitative intermediaries in agricultural trade. Traditional agricultural supply chains leave farmers vulnerable to opaque pricing and delayed payments. CropDeal bridges this divide through a cloud-native digital marketplace providing transparent pricing, live Mandi market benchmarks (MSP), competitive bidding auctions, and bilateral price negotiations.

> **Architectural Foundation:** CropDeal operates across **15 independent Spring Boot microservices**. Synchronous HTTP requests enter through a centralized **Spring Cloud API Gateway** utilizing Eureka client-side load balancing (`lb://`). Asynchronous workflows (crop alerts, bid updates, negotiations, orders, delivery tracking) are decoupled using a **RabbitMQ event bus**.

Platform Microservices Directory & Port Mappings

Service Name	Port	Primary Domain Responsibility	Database Schema	Messaging Role
api-gateway	:8080	Single edge entrypoint, route predicates, JWT auth, CORS, Swagger aggregation	Stateless	Client Ingress
eureka-server	:8761	Service registry, heartbeat monitoring, dynamic instance discovery	In-Memory	Governance
config-server	:8888	Centralized Git/file properties repository for all 15 services	Git / File Repo	Central Config
auth-service	:8081	Authentication, JWT token generation/validation, role-based access control	MySQL (auth_db)	Auth Events
user-service	:8082	Farmer, Dealer, and Delivery Partner KYC profiles and reviews	MySQL (user_db)	Profile Events
crop-service	:8083	Crop catalog, commodity types, quantity, state/district filters, status	MySQL (crop_db)	Publish: Crop Posted
price-service	:8084	National Mandi daily market rates sync, MSP benchmarks, price trends	MySQL (price_db)	Price Sync
price-alert-service	:8094	Buyer requests, target threshold subscriptions, automated match triggers	MySQL (price_alert_db)	Publish: Price Match
negotiation-service	:8085	Direct bilateral price bargaining, counter-offers, contract lock	MySQL (negotiation_db)	Publish: Neg Offer
bidding-service	:8086	Competitive auction listings, real-time bid validation, winner resolution	MySQL (bidding_db)	Publish: Bid Placed
order-service	:8088	Central order lifecycle orchestration (Direct, Auction, Negotiation)	MySQL (order_db)	Publish: Order State
payment-service	:8089	Transaction settlement, external gateways, automated refunds	MySQL (payment_db)	Publish: Paid
wallet-service	:8087	Digital wallet, double-spend prevention, escrow fund reservation & release	MySQL (wallet_db)	Escrow Events
invoice-service	:8090	Automated GST/tax calculation, itemized line billing, PDF bill generator	MySQL (invoice_db)	Consume: Paid
delivery-service	:8091	Fleet assignment, transit tracking, secure OTP delivery handshake	MySQL (delivery_db)	Publish: Delivery
notification-service	:8092	RabbitMQ consumer, omnichannel alerts (in-app, email, push)	MySQL (notification_db)	Consume: All Queues
chatbot-service	:8096	Agricultural advisory AI assistant, crop queries, FAQ automated responses	In-Memory Knowledge	Advisory Chat
report-service	:8095	Farmer earnings, dealer volume, commodity yield, platform revenue metrics	Reporting Views	Analytics Sync


CROPDEAL MICROSERVICES PLATFORM
System Architecture & Runtime Communication Topology | Spring Boot 3.3.13 • Spring Cloud 2023.0.6 • Docker Compose
Eureka Discovery
Spring Cloud Gateway
RabbitMQ Bus
Distributed MySQL
Angular 17 UI
1. PRESENTATION & CLIENT CONSUMER LAYER
Farmer Portal (Angular)
Crop catalog, bids review, negotiations
:4200/farmer
Dealer Portal (Angular)
Browse crops, live auctions, payments
:4200/dealer
Delivery App (Mobile/Web)
Order dispatch, OTP delivery proof
:4200/delivery
Admin Console (Angular)
Platform audit, user KYC, reports
:4200/admin
Swagger UI Docs
OpenAPI aggregated interactive console
:8080/swagger-ui.html
HTTPS / REST (Single Entry Point)
2. EDGE GATEWAY & SERVICE GOVERNANCE INFRASTRUCTURE
SPRING CLOUD API GATEWAY (Port 8080)
✔ Dynamic lb:// Service Routing
✔ JWT Security & Token Verification
✔ OpenAPI Docs Aggregation Hub
✔ CORS & Rate Limiting Filter
Registry Lookup
EUREKA SERVICE DISCOVERY
Spring Cloud Netflix (Port 8761)
• Client-Side Round-Robin LB
• Real-time Heartbeat Monitor
• Auto-Registration of 15 Services
DYNAMIC REGISTRY
CONFIG SERVER
Spring Cloud Config (Port 8888)
• Git/File Property Storage
• Runtime Environment Profiles
• Single Source of Truth
100% CENTRALIZED CONFIG
lb://service-id (Load Balanced REST)
3. BUSINESS MICROSERVICES DOMAIN MESH (15 DEDICATED DOMAINS)
IDENTITY & USERS
auth-service :8081
• JWT Token Provider
• Role Auth (Farmer/Dealer)
user-service :8082
• Farmer & Dealer Profiles
• Delivery Partner KYC
CATALOG & MARKET PRICING
crop-service :8083
• Crop Postings & Units
• Status: Active/Sold
price-service :8084
• Mandi Market Prices
• Live MSP Benchmark
price-alert-service :8094
• Dealer Buying Requests
• Target Price Watcher
B2B TRADE & AUCTIONS
bidding-service :8086
• Live Auction Listings
• Real-time Bid Engine
negotiation-service :8085
• Farmer-Dealer Chat/Offer
• Counter-Offer Engine
ORDER EXECUTION & FINANCE
order-service :8088
• Direct/Bid/Neg Orders
• Order State Machine
payment-service :8089
• Gateway/Wallet Interop
• Transaction Audit
wallet-service :8087
• Escrow Fund Hold/Release
• Balance Credit/Debit
invoice-service :8090
• Automated Tax/GST Calc
• Itemized Line Items
LOGISTICS & INTELLIGENCE
delivery-service :8091
• Delivery Assignment
• OTP Proof of Delivery
notification-service :8092
• RabbitMQ Consumer
• In-App/Email/SMS Alerts
chatbot-service :8096
• Advisory AI Assistant
• Crop Query Resolution
report-service :8095
• Revenue & Sales Analytics
• Performance Reports
Publish Events
Consume Events
4. ASYNCHRONOUS EVENT-DRIVEN MESSAGING BACKBONE (RabbitMQ Broker :5672 / :15672)
cropdeal.exchange
Direct & Topic Exchanges
AMQP BROKER
notification.queue
Real-time push, email, SMS notifications
price.alert.queue
Price matches against farmer crop postings
order.events.queue
Order placement, payment success, fulfillment
delivery.status.queue
Driver dispatch & real-time delivery status updates
5. RELATIONAL DATA PERSISTENCE LAYER (Containerized MySQL Database Engine :3306)
auth_db
Users, Roles, Providers
user_db
Farmers, Dealers, KYC
crop_db
Crops, Varieties, Units
bidding_db
Listings, Bids, Rules
negotiation_db
Offers, Counter-Offers
order_db
Orders, Status Machine
payment_db
Payments, Refunds
wallet_db
Balances, Escrow Holds
invoice_db
Invoices, Tax Items
delivery_db
Deliveries, Fleet Tracking

2. Architectural Breakdown & Deep Dive Explanation

2.1 Edge Layer: API Gateway & Traffic Ingress

The **API Gateway (Port 8080)** acts as the single reverse proxy and traffic guard for the entire ecosystem. Rather than exposing internal container ports directly to the outside world, external clients (Angular web application, mobile browsers) interact exclusively with port 8080. The Gateway provides four critical enterprise capabilities:

• **Dynamic Load-Balanced Routing:** Route predicates forward incoming path patterns (e.g. /api/crops/**, /api/orders/**) to microservices using the logical lb://service-name syntax resolved via Spring Cloud LoadBalancer.
• **Centralized JWT Security Filter:** Validates cryptographic signatures of Bearer tokens before forwarding requests down to internal services.
• **Unified CORS Configuration:** Enforces allowed HTTP origins, methods, and headers at the boundary, preventing browser cross-origin blocks.
• **OpenAPI Specification Aggregator:** Aggregates Swagger documentation from all 15 microservices into a single unified interactive console at /swagger-ui.html.

2.2 Service Governance: Eureka Discovery & Centralized Config

CropDeal uses **Netflix Eureka (Port 8761)** for dynamic service registration. When each microservice boots inside its Docker container, it registers its container IP and port with Eureka. The Gateway queries Eureka periodically to maintain an up-to-date registry, eliminating static IP hardcoding. Parallel to Eureka, the **Spring Cloud Config Server (Port 8888)** provides 100% centralized property management. Individual services do not maintain duplicate local configuration files; instead, they pull datasource credentials, RabbitMQ bindings, and Eureka URLs dynamically upon startup.

2.3 Asynchronous Event Backbone: RabbitMQ Message Broker

To guarantee high throughput and decouple time-sensitive transactions from notification delivery, CropDeal integrates **RabbitMQ (Port 5672)**. When business milestones occur, services immediately publish lightweight domain events to cropdeal.exchange without blocking the HTTP thread:

• **Crop Posting Event:** Emitted by crop-service → Consumed by price-alert-service and notification-service to alert subscribed dealers.
• **Bidding & Negotiation Events:** Emitted on each counter-offer or bid increment → Instantly dispatches in-app notifications to both parties.
• **Order Lifecycle Events:** Triggers invoice generation, wallet escrow reservations, and logistics fleet dispatch asynchronously.
• **Delivery Status Event:** Emitted by delivery partners on transit milestones → Triggers dealer notifications and releases escrow funds upon final OTP verification.

2.4 Persistence Strategy: Distributed Database Isolation

Following microservices architectural best practices, CropDeal avoids a monolithic shared database. Each microservice manages its own logical schema (e.g., crop_db, order_db, wallet_db). Cross-service relationships are established using immutable soft identifiers (such as farmer_id, dealer_id, crop_id) rather than direct SQL cross-schema joins. This prevents cascade locks, enables independent horizontal scaling, and allows individual database schemas to evolve independently.

CROPDEAL DOMAIN CLASS MODEL & OBJECT ARCHITECTURE

UML Class Specifications: Domain Aggregates, Layered Contracts & Microservice Boundary Collaborations

STANDARD 4-TIER MICROSERVICE DESIGN PATTERN & RESPONSIBILITY SEPARATION

«Controller» REST Controller

+ handleRequest(@Valid DTO): ResponseEntity
+ validateSecurityToken(JWT): Principal
+ publishSwaggerOpenAPI(): OpenAPI

Handles HTTP routing, parameter binding, request validation, and security authorization.

«Service» Service Implementation

+ executeBusinessRules(Command): Result
+ @Transactional commitStateChange()
+ publishDomainEvent(RabbitTemplate)

Encapsulates core business transactions, domain events publishing, and validations.

«Repository» Spring Data Repository

+ findById(ID): Optional<Entity>
+ save(Entity): Entity
+ findByCustomCriteria(JPQL): List<T>

Data access abstraction providing CRUD and optimized derived/native SQL query operations.

«Entity» Domain Entity Model

- id: Long {id, generated}
- auditFields: CreatedAt, UpdatedAt
+ getters(), setters(), businessMethods()

Persistent domain entity mapping to relational tables with optimistic concurrency control.



3. Domain Class Model & Collaboration Specifications

3.1 4-Tier Software Architecture & Design Patterns

Every microservice within CropDeal adheres strictly to an enterprise layered architecture pattern:
1. **REST Controller Layer (@RestController):** Defines clean RESTful HTTP endpoints, validates client payloads using Bean Validation (@Valid, @NotNull), enforces role-based security (@PreAuthorize), and documents schemas for OpenAPI/Swagger.
2. **Service Business Logic Layer (@Service):** Encapsulates core business transactions, boundary checks, and state transitions. Methods modifying data state are marked with @Transactional to guarantee ACID atomicity. It acts as the event publisher via RabbitTemplate.
3. **Spring Data Repository Layer (@Repository):** Extends JpaRepository<Entity, ID>, offering derived query methods and optimized JPQL projections without boilerplate SQL.
4. **Domain Entity Layer (@Entity):** Represents relational database tables mapped through Hibernate JPA with explicit audit timestamps and validation annotations.

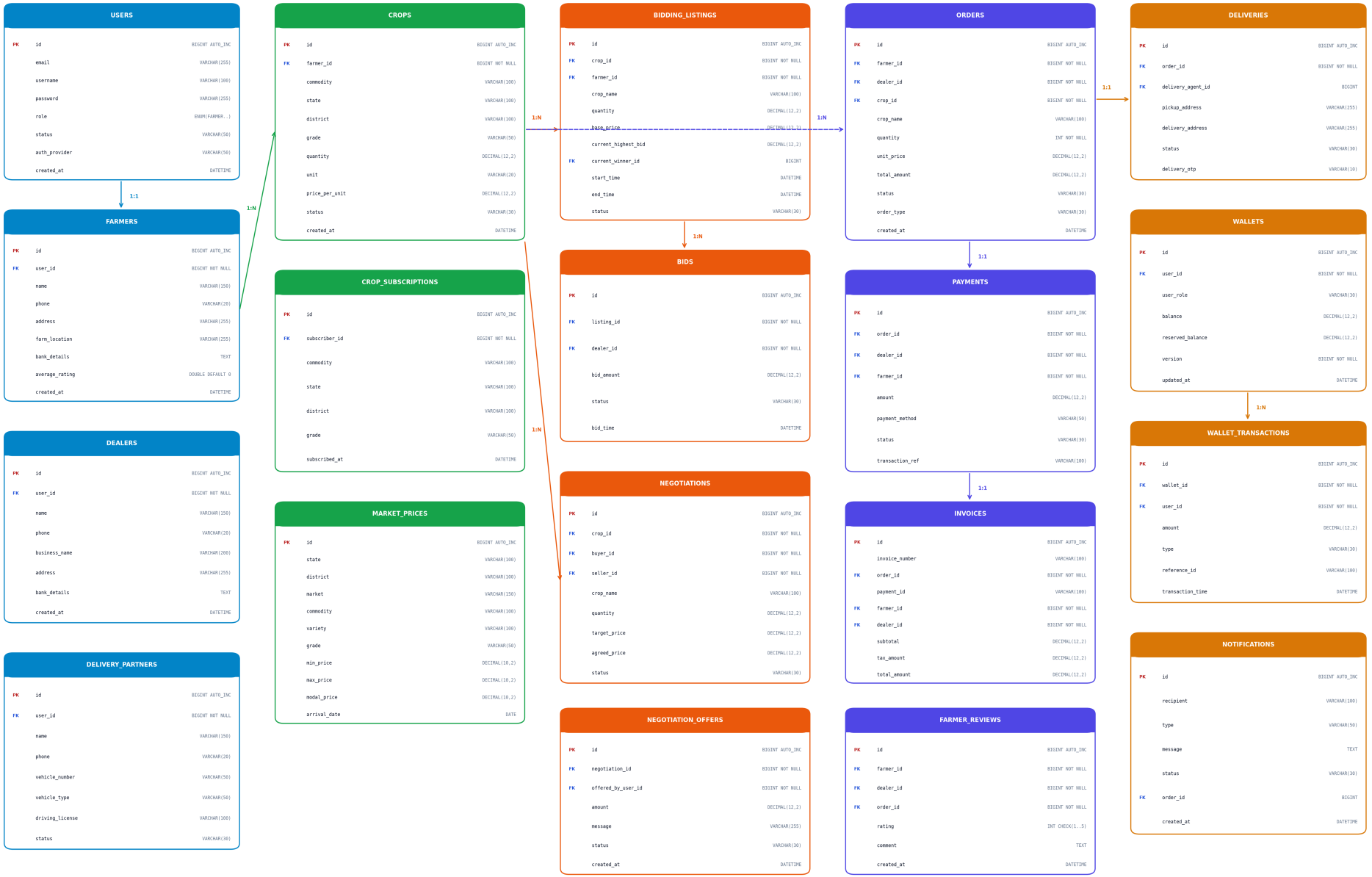
3.2 Key Domain Aggregates & Class Relationships

Domain Class	Key Field Members	Core Methods & Operations	Class Relationships & Cardinality
Crop	farmerId, commodity, state, district, grade, quantity, pricePerUnit, status	+ isAvailable(): Boolean + reserveQuantity() + markAsSold()	Belongs to Farmer (N:1). Associated with BiddingListing (1:1) and Negotiations (1:N).
BiddingListing & Bid	cropId, farmerId, basePrice, currentHighestBid, currentWinnerId, startTime, endTime	+ placeBid(Bid): Boolean + closeAuction(): Long + markAsOutbid()	BiddingListing is aggregate root composing 1..* Bid instances. Associates with winning Dealer.
Negotiation & Offer	cropId, buyerId, sellerId, targetPrice, agreedPrice, status, offers: List<Offer>	+ addCounterOffer() + acceptOffer() + rejectNegotiation()	Negotiation manages a collection of 1..* NegotiationOffer instances representing counter-offers.
Order	farmerId, dealerId, cropId, quantity, unitPrice, totalAmount, status, orderType	+ calculateTotal() + confirmOrder() + markDelivered() + cancelOrder()	Central commercial hub. Originates 1:1 Payment, 1:1 Invoice, and 1:1 Delivery.
Wallet & Escrow	userId, userRole, balance, reservedBalance, version (optimistic lock)	+ creditBalance() + debitBalance() + reserveEscrow() + releaseEscrow()	Wallet aggregates 1..* WalletTransaction and 1..* WalletReservation records.
Delivery & Fleet	orderId, deliveryAgentId, pickupAddress, deliveryAddress, status, deliveryOtp	+ assignAgent() + updateStatus() + verifyOtpAndComplete()	Delivery aggregates 1..* DeliveryStatusHistory and 1..* DeliveryAssignment.

CROPDEAL RELATIONAL DATABASE ENTITY SPECIFICATIONS (ER DIAGRAM)

Entity Schemas, Relational Primary/Foreign Keys, Data Typing & Cardinalities Across Distributed Microservice Schemas

[PK] Primary Key [FK] Foreign Key [1:1] One-to-One



4. Database Schema Design & Data Integrity Guidelines

4.1 Relational Schema Organization & Key Tables

CropDeal's relational architecture balances relational integrity with microservice autonomy. The schema dictionary is structured across the following primary tables:

- **users:** Central authentication record storing email, bcrypt hashed password, and role.
- **farmers & dealers:** Extended role-specific profiles linked to users.id via 1:1 foreign key.
- **crops:** Agricultural listings detailing commodity, agricultural grade, quantity in metric units, state, district, and base price.
- **bidding_listings & bids:** Auction listings linked to crops; bids enforce 1:N cardinality with descending order ranking.
- **negotiations & negotiation_offers:** Bilateral chat negotiations tracking agreed price targets.
- **orders:** The central fulfillment record storing quantity, locked unit price, calculated total, and state machine status.
- **payments & invoices:** Financial audit trail, transaction reference hashes, and itemized tax breakdowns.
- **deliveries:** Logistics dispatch tracking pickup/drop coordinates, delivery agent assignment, and OTP verification code.
- **wallets & wallet_transactions:** Digital wallet balances, ledger entries, and escrow hold reservations.

4.2 Enterprise Financial Precision & Concurrency Control

To avoid the catastrophic floating-point rounding errors common with standard FLOAT or DOUBLE types in banking applications, **all monetary amounts, unit rates, quantities, and bids are strictly modeled as DECIMAL(12,2) in MySQL and BigDecimal in Java.**

Optimistic Concurrency Control (@Version): In high-concurrency environments—such as rapid auction bids or simultaneous order placements—there is a risk of double-spending wallet balances. CropDeal equips the wallets table with an explicit version BIGINT NOT NULL column. Hibernate enforces optimistic locking: if two transactions attempt to reserve or debit the same wallet simultaneously, the second transaction detects a version conflict, rolls back safely, and throws an OptimisticLockException, preventing balance corruption.

4.3 Eventual Consistency & Distributed Transactions (Saga Pattern)

Because CropDeal spans multiple independent databases, traditional distributed two-phase commits (2PC) are avoided due to latency and locking bottlenecks. Instead, the platform implements an **Event-Driven Choreographed Saga**:

1. When an order is created (status = CREATED), order-service publishes an OrderCreatedEvent.
2. wallet-service consumes the event and locks funds in escrow (status = ESCROW_RESERVED).
3. Upon payment confirmation, delivery-service initiates driver dispatch.
4. If any step fails (e.g. insufficient wallet balance or cancelled order), a compensating event is dispatched, releasing the reserved funds and transitioning the order to CANCELLED.

5. Business Workflows & Presentation Cheat Sheet

Use this structured cheat sheet during technical reviews, professor evaluations, or engineering interviews to clearly explain CropDeal's end-to-end operation.

Commerce Flow	Step-by-Step Execution Sequence	Involved Services & Protocols
1. Direct Crop Purchase	1. Farmer posts crop listing with fixed price per unit. 2. Dealer discovers crop via Gateway search. 3. Dealer clicks 'Buy Now' → Order created in CREATED state. 4. Dealer pays via Escrow Wallet → Funds reserved. 5. Delivery partner assigned → Picks up crop from farm. 6. Delivery verified with 6-digit OTP → Funds credited to Farmer.	crop-service (REST) order-service (REST) wallet-service (Escrow) delivery-service (OTP) notification-service (RabbitMQ)
2. Competitive Live Auction	1. Farmer lists crop for auction with base reserve price and deadline. 2. Dealers place escalating bids; wallet verifies reserve ability. 3. Outbid dealers receive instant RabbitMQ notifications. 4. Countdown timer expires → Highest valid bid declared winner. 5. System auto-generates Order with locked winning bid price.	bidding-service (REST) wallet-service (Hold) order-service (Auto-Create) RabbitMQ (bid.events.queue)
3. Counter-Offer Price Negotiation	1. Dealer reviews crop and proposes lower counter-offer rate. 2. Farmer receives push notification and reviews offer. 3. Farmer can Accept, Reject, or submit a revised Counter-Offer. 4. Both parties agree on target price → Negotiation locked. 5. Order created automatically with agreed negotiated price.	negotiation-service (REST) order-service (REST) RabbitMQ (negotiation.events.queue)
4. Smart Mandi Price & Alert Match	1. Mandi API syncs live wholesale crop prices across districts. 2. Dealers subscribe to price alert thresholds for specific commodities. 3. When farmer lists crop matching dealer's target rate, RabbitMQ dispatches instant notification.	price-service (Sync) price-alert-service (Match) RabbitMQ (price.alert.queue)

Quick Summary for Interview / Presentation:

- **Architecture:** 15 Spring Boot 3.3.13 microservices running on Java 21, Spring Cloud Gateway (:8080), Netflix Eureka (:8761), and Spring Cloud Config (:8888).
- **Messaging:** Event-driven architecture with RabbitMQ decoupled topic exchanges and queues for non-blocking notifications.
- **Security:** Centralized JWT authentication filter and role-based access control (Farmer, Dealer, Delivery Partner, Admin).
- **Data Consistency:** Distributed database-per-service pattern with optimistic concurrency locking (@Version) and choreographed saga workflows.